Lecture 26.10: Logistic Regression from Scratch — Vectorized Implementation & Decision Boundaries

📌 Executive Summary

Vectorized NumPy OOP implementation of LogisticRegression, probability estimation, thresholding, decision boundary geometry, and live testing on lecture datasets.

🔬 Mathematical Formulation & Theory

1. Vectorized OOP Implementation

Following the exact curriculum architecture presented in the lecture video:

```
class LogisticRegression:
    def __init__(self, learning_rate=0.01, n_iter=1000):
        self.bias = None
        self.weights = None
        self.lr = learning_rate
        self.n_iter = n_iter

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
        self.bias = 0.0
        self.weights = np.zeros(n)

        for i in range(self.n_iter):
            z = self.bias + np.dot(X, self.weights)
            y_pred = self._sigmoid(z)

            db = (1.0 / m) * np.sum(y_pred - y)
            dw = (1.0 / m) * np.dot(X.T, (y_pred - y))

            self.bias -= self.lr * db
            self.weights -= self.lr * dw

    def get_probabilities(self, X):
        z = self.bias + np.dot(X, self.weights)
        return self._sigmoid(z)
```

```
    def predict(self, X, threshold=0.5):
        probabilities = self.get_probabilities(X)
        y_pred_bool = probabilities >= threshold
        return y_pred_bool.astype(int)
```

2. Decision Boundary Geometry

The decision boundary separates class 0 from class 1 at probability threshold $P(y=1|\mathbf{x}) = 0.5$:

$$\sigma(z) = 0.5 \iff z = \mathbf{w}^T\mathbf{x} + b = 0$$

In a two-dimensional feature space (x_1, x_2):

$$w_1x_1 + w_2x_2 + b = 0 \implies x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

This equation defines a straight separating line with slope $m = -\frac{w_1}{w_2}$ and intercept $c = -\frac{b}{w_2}$.

💻 Python & NumPy Implementation

```
import numpy as np

class LogisticRegression:
    def __init__(self, learning_rate=0.01, n_iter=1000):
        self.bias = None
        self.weights = None
        self.lr = learning_rate
        self.n_iter = n_iter

    def _sigmoid(self, z):
        return 1.0 / (1.0 + np.exp(-z))

    def fit(self, X, y):
        m, n = X.shape
        self.bias = 0.0
        self.weights = np.zeros(n)

        for i in range(self.n_iter):
            z = self.bias + np.dot(X, self.weights)
            y_pred = self._sigmoid(z)
```

```
            db = (1.0 / m) * np.sum(y_pred - y)
            dw = (1.0 / m) * np.dot(X.T, (y_pred - y))

            self.bias -= self.lr * db
            self.weights -= self.lr * dw

    def get_probabilities(self, X):
        z = self.bias + np.dot(X, self.weights)
        return self._sigmoid(z)

    def predict(self, X, threshold=0.5):
        probabilities = self.get_probabilities(X)
        y_pred_bool = probabilities >= threshold
        return y_pred_bool.astype(int)

# Lecture Dataset
X = np.array([[1, 2], [2, 3], [3, 4], [4, 5]])
y = np.array([0, 0, 1, 1])

model = LogisticRegression(learning_rate=0.1, n_iter=2000)
model.fit(X, y)

print("Predictions:  ", model.predict(X))
print("Probabilities:", model.get_probabilities(X))
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.